

COM2002-1 Programación de Software de Sistemas**Profesor :** Joaquín Soto**Ayudante:** Arnaldo González

Ayudantía de Repaso

Punteros, strings, memoria dinámica y árboles

Las preguntas P1 y P2 son de programación en C: explicita sus supuestos y use funciones auxiliares si le ayudan a ordenar la solución. La P3 es conceptual y se responde en pocas líneas.

P1. Escriba una función eficiente `void invertir\_palabras(char *frase)` que reciba una frase (un arreglo de caracteres terminado en `'\0'`) y deje, en el mismo arreglo, sus palabras en orden inverso. Las letras de cada palabra deben conservar su orden. Por ejemplo:

"el proceso padre espera al hijo" $\rightarrow$ "hijo al espera padre proceso el"

Su solución no puede usar un arreglo auxiliar ni pedir memoria dinámica. Indique cómo vienen separadas las palabras según sus supuestos y cuál es el orden de complejidad de su implementación, en tiempo y en memoria adicional.

Indicación: ¿qué obtiene si primero invierte la frase completa, carácter por carácter?

Bonus: usando la misma idea, escriba `void rotar(char *s, int k)`, que desplace los caracteres de `s` en `k` posiciones hacia la derecha de forma circular, en tiempo lineal y sin arreglo auxiliar. Por ejemplo, al rotar "sistemas" en 3 posiciones se obtiene "massiste".

P2. Considere la función `strins()`. No forma parte de `string.h`, pero su documentación sigue el mismo formato que las páginas del manual:

SINOPSIS

```
char *strins(char *dest, const char *src, size_t pos);
```

DESCRIPCIÓN

La función `strins()` inserta la cadena `src` dentro de la cadena `dest`, a partir de la posición `pos` (la primera posición es la 0). Los caracteres de `dest` que estaban desde `pos` en adelante se desplazan hacia la derecha para hacer espacio, y el resultado queda terminado en `'\0'`. Si `pos` es mayor que el largo de `dest`, `src` se agrega al final. Las cadenas no deben solaparse, y `dest` debe tener espacio suficiente para el resultado.

VALOR DEVUELTO

La función `strins()` devuelve un puntero a la cadena resultante `dest`.

Por ejemplo, si `dest` contiene "free(): pointer":

```
strins(dest, "invalid ", 8)  $\rightarrow$  "free(): invalid pointer"
```

- a) Implemente `strins()` sin usar funciones de `string.h`. Puede escribir las funciones auxiliares que necesite.
- b) Un compañero implementó el desplazamiento recorriendo `dest` de izquierda a derecha, es decir, desde `pos` hacia el final. ¿Funciona su versión con el ejemplo anterior? Encuentre un caso en que falle y explique qué ocurre en memoria.

Bonus: escriba `char *strins2(char *dest, const char *src, size_t pos)`, que haga lo mismo que `strins()` pero sin suponer que `dest` tiene espacio suficiente: cuando haga falta debe pedir más memoria, y en ese caso el puntero retornado puede ser distinto de `dest`. Explícite qué necesita suponer sobre la memoria de `dest` y muestre cómo se debería llamar a `strins2()` desde `main`.

P3. Preguntas conceptuales. Responda en pocas líneas. Cuando se pregunte por tamaños, suponga una máquina de 64 bits, con `int` de 4 bytes y punteros de 8 bytes.

- a) Dentro de una función se declara:

```
char a[] = "kernel";
char *p = "kernel";
```

¿Qué valores entregan `sizeof(a)`, `sizeof(p)`, `strlen(a)` y `strlen(p)`? De las instrucciones `a[0] = 'K'` y `p[0] = 'K'`, ¿cuál es válida y qué problema tiene la otra?

- b) Si se define la función de abajo y se llama con `f(a)`, ¿qué imprime? ¿Qué consecuencia tiene esto para `strins()`, que recibe `dest` sin su tamaño?

```
void f(char s[]) { printf("%zu\n", sizeof(s)); }
```

Por otro lado, ¿qué impide el `const` de `const char *src`? ¿Es válido hacer `src++` dentro de la función? Compare con `char *const src`.

- c) La siguiente función es correcta. ¿Cuál es su complejidad y por qué no es eficiente?

```
void a_mayusculas(char *s)
{
    for (size_t i = 0; i < strlen(s); i++)
        if (s[i] >= 'a' && s[i] <= 'z')
            s[i] = s[i] - 'a' + 'A';
}
```

Algo parecido ocurre al unir `k` palabras llamando a `strcat(buf, palabras[i])` dentro de un ciclo. Explique por qué el costo total crece de forma cuadrática y cómo lo evitaría.

- d) Considere la siguiente estructura:

```
struct nodo {
    int valor;
    struct nodo *izq;
    struct nodo *der;
};
```

¿Por qué `izq` y `der` tienen que ser punteros y no de tipo `struct` `nodo`? ¿Cuánto espera que valga `sizeof(struct nodo)` y por qué no es simplemente $4 + 8 + 8 = 20$? Si `r` es de tipo `struct nodo *`, ¿qué diferencia hay entre `r->izq`, `(*r).izq` y `*r.izq`?

e) Encuentre el error de cada fragmento y corrijalo.

I. Copiar una cadena en memoria nueva:

```
char *duplicar(const char *s)
{
    char *d = malloc(strlen(s));
    strcpy(d, s);
    return d;
}
```

II. Agrandar un bloque para concatenar:

```
char *s = malloc(5);
strcpy(s, "hola");
realloc(s, 11);
strcat(s, " mundo");
```

III. Crear un nodo de un árbol:

```
struct nodo *nuevo_nodo(int v)
{
    struct nodo n = { v, NULL, NULL };
    return &n;
}
```

f) ¿Qué calcula la siguiente función? Luego escriba `void liberar(struct nodo *r)`, que libere todos los nodos de un árbol, e indique en qué orden debe recorrerlo y por qué.

```
int misterio(struct nodo *r)
{
    if (r == NULL)
        return 0;
    if (r->izq == NULL && r->der == NULL)
        return 1;
    return misterio(r->izq) + misterio(r->der);
}
```

g) Considere expresiones lógicas formadas por variables de una letra y los operadores `!` (negación), `&&` (y) y `||` (o). La negación es unaria, tiene la mayor precedencia y se asocia de derecha a izquierda (`!!a` equivale a `!(!a)`); después viene `&&` y al final `||`, ambos binarios y asociativos de izquierda a derecha. Estas expresiones se pueden representar con un árbol binario cuyas hojas son variables y cuyos nodos internos son operadores.

- I. Dibuje el árbol de `!a || b && !c || d`. ¿Cuál es la raíz y qué nodos tienen un solo hijo?
- II. Escriba el recorrido en postorden del árbol. ¿Por qué es el orden adecuado para evaluar la expresión?
- III. Enuncie una regla para decidir qué operador va en la raíz de una expresión sin paréntesis. A partir de ella, describa, sin escribir código, un algoritmo recursivo que construya el árbol a partir del texto.
- IV. ¿Cómo cambia su regla si se permiten paréntesis? Dibuje el árbol de `!(a || b) && c`.